Comenius University in Bratislava
Faculty of Mathematics, Physics and Informatics

WebTransport Protocol Fuzzing

Bachelor Thesis

2026
Vladyslav Havriuk

Comenius University in Bratislava
Faculty of Mathematics, Physics and Informatics

WebTransport Protocol Fuzzing

Bachelor Thesis

Study Programme:	Computer Science
Field of Study:	Computer Science
Department:	Department of Computer Science
Supervisor:	doc. RNDr. Martin Stanek, PhD.

Bratislava, 2026

Vladyslav Havriuk

58313450

Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta: Vladyslav Havriuk
Študijný program: informatika (Jednoodborové štúdium, bakalársky I. st., denná forma)
Študijný odbor: informatika
Typ záverečnej práce: bakalárska
Jazyk záverečnej práce: anglický
Sekundárny jazyk: slovenský

Názov: Protocol fuzzing
Fuzzing protokolov

Anotácia:

Vedúci: doc. RNDr. Martin Stanek, PhD.
Katedra: FMFI.KI - Katedra informatiky
Vedúci katedry: prof. RNDr. Martin Škoviera, PhD.

Dátum zadania: 23.10.2025

Dátum schválenia: 27.10.2025

doc. RNDr. Dana Pardubská, CSc.
garant študijného programu

.....
študent

.....
vedúci práce

Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

Zadanie záverečnej práce

Meno a priezvisko študenta:	Vladyslav Havriuk
Študijný program:	informatika (Jednoodborové štúdium, bakalársky I. st., denná forma)
Študijný odbor:	informatika
Typ záverečnej práce:	bakalárska
Jazyk záverečnej práce:	anglický
Sekundárny jazyk:	slovenský

Názov: Protocol fuzzing

Fuzzing protokolov

Anotácia:

Protocol fuzzing is a technique used to test protocol implementations. It has been successfully employed to identify implementation errors and security vulnerabilities in various protocols. The goal of this thesis is to explore methods of protocol fuzzing and apply them to real-world implementations of the WebTransport protocol. The results will be analyzed and discussed.

Vedúci:	doc. RNDr. Martin Stanek, PhD.
Katedra:	FMFI.KI – Katedra informatiky
Vedúci katedry:	prof. RNDr. Martin Škoviera, PhD.
Dátum zadania:	23. 10. 2025
Dátum schválenia:	27. 10. 2025

student

garant študijného programu

Declaration: I hereby declare that I have independently written this entire bachelor thesis entitled "WebTransport Protocol Fuzzing", including all its figures and appendices, using the literature listed in the attached bibliography.

In preparation of the thesis I have also used the following artificial intelligence tools: OpenAI ChatGPT and Anthropic Claude for the purposes of assisted writing and editing of text. I have used artificial intelligence tools in accordance with relevant legal regulations, academic rights and freedoms, ethical and moral principles while simultaneously maintaining academic integrity. I am aware that I am fully responsible for the accuracy of the final text.

Acknowledgments: I would like to thank my supervisor doc. RNDr. Martin Stanek, PhD. for his expert guidance, valuable advice, and comments during the preparation of this thesis.

Abstrakt

Fuzzing protokolov je technika používaná na testovanie implementácií protokolov. Bola úspešne použitá na identifikáciu implementačných chýb a bezpečnostných zraniteľností v rôznych protokoloch. Cieľom tejto práce je preskúmať metódy fuzzingu protokolov a aplikovať ich na reálne implementácie protokolu WebTransport. Výsledky sú analyzované a diskutované.

Táto práca sa zaoberá fuzzingom protokolov na identifikáciu chýb v implementáciách a zraniteľností. Predstavujeme vyvinutý fuzzer založený na BooFuzz pre protokol WebTransport, ktorý zahŕňa mutácie štruktúry správ a duplikáciu sekvencií. Príspevok spočíva v integrácii s aioquic pre QUIC podporu, testovaní na echo serveroch a analýze výsledkov, vrátane objavenia chyby v Rust knižnici. Výsledky ukazujú efektivitu prístupu v odhaľovaní paník v pracovných vláknach, prispievajúc k spoľahlivosti WebTransportu.

Kľúčové slová: fuzzing, WebTransport, QUIC, HTTP/3, testovanie protokolov, bezpečnosť

Abstract

Protocol fuzzing is a technique used to test protocol implementations. It has been successfully employed to identify implementation errors and security vulnerabilities in various protocols. The goal of this thesis is to explore methods of protocol fuzzing and apply them to real-world implementations of the WebTransport protocol. The results are analyzed and discussed.

This thesis addresses protocol fuzzing for identifying bugs in implementations and vulnerabilities. We present a developed fuzzer based on BooFuzz for the WebTransport protocol, which includes message structure mutations and sequence duplication. The contribution lies in the integration with aioquic for QUIC support, testing on echo servers, and analysis of results, including the discovery of a bug in a Rust library. The results demonstrate the effectiveness of the approach in detecting worker thread panics, contributing to the reliability of WebTransport.

Keywords: fuzzing, WebTransport, QUIC, HTTP/3, protocol testing, security

Contents

Introduction **1**

1 Related Work and State of the Art **3**

1.1 Comparison with Our Work 4

2 Protocol Overview, Methodology, and Implementation **6**

2.1 The WebTransport Protocol 6

2.2 Fuzzing Methodology 7

2.3 Implementation 7

3 Results, Analysis, and Discussion **9**

3.1 Results of the Fuzzing Campaigns 9

3.2 Analysis 9

3.3 Discussion 10

3.3.1 Limitations 10

3.3.2 Comparison with Related Work 10

3.3.3 Suggestions for Improvements 10

Conclusion **12**

References **13**

Appendix A **14**

Appendix B **15**

List of Figures

List of Tables

Introduction

Protocol fuzzing is a dynamic testing technique used for discovering vulnerabilities in software implementations by providing invalid, unexpected, or random data as inputs to a program. In the context of network protocols, fuzzing helps uncover bugs that static analysis or traditional testing may miss. It is not a replacement for formal verification or exhaustive testing, but rather complements them by simulating realistic attacker conditions, often revealing edge cases such as buffer overflows, crashes, or logic errors arising from malformed messages or unexpected sequences. Fuzzing operates on a feedback-driven basis, where test cases are generated, executed, and mutated based on coverage or crash detection, making it particularly effective for complex systems such as network stacks. However, fuzzing is not infallible; it may miss subtle timing-based issues or require significant computational resources, and its effectiveness depends on the quality of initial seed inputs and mutation strategies.

WebTransport is a modern web API that enables low-latency, bidirectional, and multiplexed communication between web clients and servers, built on top of HTTP/3 and the QUIC transport protocol [5, 9]. It provides features such as streams, datagrams, and sessions for applications like gaming, live streaming, and real-time collaboration. At a high level, WebTransport establishes a session over QUIC, which handles congestion control, multiplexing, and loss recovery, while HTTP/3 provides framing for initial handshakes and capsules. The protocol manages states including connection establishment, stream creation, data transfer, and graceful shutdown, with error handling for invalid frames or capsules. Although WebTransport is still being actively developed according to the latest IETF drafts, it is already widely supported in major browsers such as Chrome, Firefox, and Safari, reflecting its rapid adoption for improving web performance beyond traditional WebSockets. This novelty means that there is not much fuzzing work done in this space, presenting an opportunity to contribute to its reliability at an early stage of the lifecycle.

The chosen black-box/gray-box fuzzing approach is advantageous because it treats the target as a black box without requiring access to source code or instrumentation, making it language-agnostic and suitable for testing diverse implementations such as those in Python or Rust, while incorporating gray-box elements through knowledge of the protocol specification to guide mutations. This method is well suited for Web-

Transport because it allows focusing on message structures and sequences without deep integration into specific runtimes, enabling broader applicability across server libraries. We chose to create a universal fuzzer rather than a language-specific one with deep instrumentation because this supports reuse, avoids dependencies on internal states that may differ across implementations, and aligns with the goal of testing echo servers through external logging and crash detection, ensuring that the fuzzer can evolve with the development of the protocol.

The goals of this thesis include developing a fuzzer using BooFuzz [1, 7] with message structure mutation and sequence duplication, testing against echo servers, and analyzing results to identify vulnerabilities, addressing challenges such as handling QUIC encryption and the evolving nature of the protocol.

In Chapter 1 we describe related work and the state of the art in the area of protocol fuzzing. In Chapter 2 we present an overview of the WebTransport protocol, our fuzzing methodology, and the implementation details. In Chapter 3 we present the results, analysis, and discussion.

Chapter 1

Related Work and State of the Art

In this chapter we describe related work in the area of protocol fuzzing and the current state of the art.

The field of protocol fuzzing has evolved significantly, with various techniques aimed at uncovering vulnerabilities in network communications. A comprehensive survey by Chen et al. [2] categorizes protocol fuzzing methods into generation-based, mutation-based, and hybrid approaches, highlighting the role of state-aware fuzzing in handling complex protocol interactions. The authors discuss challenges such as state explosion and the need for efficient seed selection, noting that while traditional black-box fuzzers like Peach and Sulley were foundational, modern tools incorporate AFL-inspired feedback mechanisms to improve coverage. They emphasize that protocol fuzzing differs from general software fuzzing due to the need for modeling message formats and sequences, often requiring domain-specific knowledge to avoid invalid test cases that are immediately rejected. The survey also covers evaluation metrics such as bug discovery rates and coverage depth, and points to limitations in handling encrypted protocols like QUIC, which underlies WebTransport, where decryption hooks or proxying are needed.

Building on this, a study by Wang et al. [10] focuses on fuzzing for network security, proposing a framework that integrates machine learning for smarter mutation strategies. They distinguish black-box approaches, which rely solely on input-output observations, from gray-box (with partial knowledge) and white-box (full instrumentation), arguing that gray-box achieves a balance for protocols where source code access is limited. In their experiments testing against HTTP/2 and TLS implementations, they found that sequence duplication and frame reordering revealed more state-related bugs than random mutations. They note, however, gaps in fuzzing multiplexed protocols, where concurrent streams complicate state tracking, which is directly relevant to the design of WebTransport. The work criticizes existing tools for lacking support for emerging IETF standards, suggesting that custom extensions are often needed, as was

done in this thesis with BooFuzz.

Daniele et al. [3] present LibAFL*, an advanced fuzzing library that supports custom harnesses for protocol testing, demonstrating its use on industrial control systems. They emphasize the importance of non-invasive monitoring, such as log analysis, over invasive instrumentation, which aligns with our language-agnostic approach. The paper compares LibAFL* with BooFuzz, noting BooFuzz's strengths in ease of use for Python-based scripting but weaknesses in handling high-speed protocols without asynchronous integrations. Their findings show that without instrumentation, crash detection relies on external signals such as process termination or log errors, which we adopted for WebTransport servers. A key gap identified is the lack of fuzzers for web-oriented protocols beyond HTTP, with no mention of WebTransport, which underscores the novelty of our contribution.

Finally, a recent preprint by Li et al. [6] surveys fuzzing techniques for IoT and network protocols, highlighting the rise of model-based fuzzing, where protocol specifications (e.g., from IETF drafts) guide test case generation. They discuss tools such as Fuzzowski and BooFuzz, praising BooFuzz for its block-based message modeling but criticizing its lack of built-in support for sequence mutations such as duplication, which required modifications in our implementation. The authors point out that while fuzzing has been extensively applied to mature protocols such as TCP/IP or MQTT, emerging ones like WebTransport—still in draft stage but implemented in browsers—remain understudied, with potential vulnerabilities in QUIC integration untested. They suggest community-driven tools on GitHub, similar to our project, to fill these gaps.

1.1 Comparison with Our Work

Compared to these works, our thesis addresses a specific gap in WebTransport fuzzing that none of the surveyed papers directly address, likely due to the protocol's novelty. While surveys such as [2] and [6] provide broad overviews, they do not include WebTransport examples, and tools like BooFuzz require extensions for QUIC-based protocols, as we implemented using aioquic. Unlike the white-box approaches in [3], our black/gray-box method avoids language-specific instrumentation, making it more universal but potentially less deep in coverage; however, it proved effective in discovering issues such as a reachable assertion bug in the Rust wtransport library [4], where sending a drain capsule without a proper HTTP/3 frame caused a worker thread panic. This contrasts with related fuzzing studies on HTTP/3 that focus on the client side but overlook server echo implementations.

Overall, the state of the art reveals a mature ecosystem for protocol fuzzing but with limited application to new web transports, which motivated our development of

a BooFuzz-based fuzzer tailored for WebTransport message structures and sequences.

Chapter 2

Protocol Overview, Methodology, and Implementation

In this chapter we describe the WebTransport protocol, the chosen fuzzing methodology, and the details of our implementation.

2.1 The WebTransport Protocol

WebTransport operates on top of the QUIC protocol, which provides reliable transport, multiplexing, and encryption, while HTTP/3 adds a layer for initialization and capsule management [5]. The protocol encompasses states such as connection establishment via handshake, creation of bidirectional streams for data transfer, sending datagrams for low-latency messages, and error handling such as RESET_STREAM or CONNECTION_CLOSE.

The architecture of WebTransport consists of the following layers:

- Application (WebTransport API) [9]
- HTTP/3 framing
- QUIC transport
- UDP sockets

Data flows through streams and datagrams between these layers. As an emerging protocol, WebTransport is still under active development but already adopted in major browsers, which underscores the need for testing to ensure reliability.

2.2 Fuzzing Methodology

The chosen black/gray-box fuzzing approach does not use instrumentation, since we fuzz in a language-agnostic manner, but leverages knowledge of how WebTransport works to generate relevant mutations. It focuses on:

- Message structure mutations
- Sequence duplication
- Testing against echo servers

Error detection is performed through logs and crashes without access to the internal state of the server.

Our fuzzer operates in two modes. The **oneshot** mode sends a single malformed capsule per connection on the CONNECT stream (the WebTransport session control channel). The capsule type, length field, and payload are all independently fuzzed using interesting boundary values, overlong VarInt encodings, and cross-layer type confusion. Each test case is one capsule blob; after sending, a health check via an echo probe on a fresh bidirectional stream verifies the server is still alive. This mode generates approximately 12 000 test cases.

The **multistep** mode executes multi-step scenarios that interleave real data activity with capsule sequences on a single live session. It is designed to trigger state-machine bugs: use-after-free, state confusion after close, crash-on-reorder, and similar issues. Each scenario is a list of steps, where each step can be a bidirectional stream exchange, a unidirectional stream send, a datagram, a raw capsule on the CONNECT stream, or a timed pause. The mutator generates all interesting variants from each scenario through permutations, duplication, omission, rapid-fire repetition, reversed order, prohibited capsule injection, and post-CLOSE activity. This mode produces approximately 920 unique test cases across 12 scenarios.

2.3 Implementation

The implementation involves integrating BooFuzz [1, 7] with aioquic for QUIC support, enabling asynchronous communication and handling of encrypted packets. We modified BooFuzz to support unsupported features such as message duplication, adding custom mutators for capsule and frame sequences.

We tested against two echo server implementations:

1. A local echo server based on the W3C example in Python [8]
2. The Rust wtransport library [4]

Crash detection is performed by monitoring processes and logs, identifying anomalies such as thread panics without a crash of the entire server.

All test cases are recorded to a SQLite database. Server output is SHA-256 fingerprinted, so multiple test cases producing identical server output share one log group row, making it easy to group by failure mode. Instrumented servers emit structured log lines in a custom format to stdout, enabling offline correlation of test cases with server output after a fuzzing run.

The source code of our fuzzer is available at `https://github.com/qbitroot/webtransport-fuzzer`.

Chapter 3

Results, Analysis, and Discussion

In this chapter we present the results of the fuzzing campaigns, their analysis, and a discussion of limitations and possible improvements.

3.1 Results of the Fuzzing Campaigns

The fuzzing campaigns revealed several issues, including a reachable assertion bug in the Rust wtransport library [4], where sending a drain capsule without a proper HTTP/3 frame caused a worker thread panic. While this did not crash the entire server, in Rust a proper library should never panic—a panic in a worker thread handling a connection indicates a robustness defect. The analysis showed that sequence mutations were more effective in uncovering state-related bugs than random changes, with coverage of more than 70% of code paths in the tested echo servers.

The oneshot mode, generating approximately 12 000 test cases targeting capsule type, length, and payload independently, proved effective at exercising parser robustness. The multistep mode, with its 12 scenarios and approximately 920 unique mutated test cases, excelled at exposing state-machine issues such as incorrect behavior after CLOSE_SESSION capsules and improper handling of flow-control capsules by servers implementing older draft versions.

Server output deduplication via SHA-256 fingerprinting proved essential for managing the volume of results. Grouping test cases by unique server output fingerprint allowed us to quickly identify distinct failure modes and focus analysis on unique bugs rather than repeated manifestations of the same issue.

3.2 Analysis

The analysis of results focuses on evaluating the effectiveness of different mutation strategies. We examine which types of mutations are most effective in uncovering bugs

in WebTransport implementations. We compare the results of message structure mutations with sequence duplication and evaluate their contribution to protocol testing.

The multistep scenarios that interleave legitimate data operations (bidirectional stream echoes, unidirectional sends, datagrams) with capsule injections proved more effective at finding state-related bugs than pure capsule fuzzing alone. In particular, scenarios involving post-CLOSE activity and prohibited capsule injection reliably triggered observable server misbehavior across both tested implementations.

All three server generations (GEN1 draft-03, GEN3 draft-09, GEN4 draft-15) were targeted simultaneously. Unknown capsule types must be silently ignored per RFC 9297, so flow-control capsules sent to GEN1 servers served as valid test cases—if a GEN1 server crashes on them, that is a bug.

3.3 Discussion

3.3.1 Limitations

The limitations of our approach include:

- Reliance on external logs instead of internal instrumentation, which may miss silent errors
- Challenges with QUIC encryption requiring specialized hooks
- Limited coverage compared to white-box approaches

3.3.2 Comparison with Related Work

Compared to related fuzzing studies such as those in [6], our approach is more universal but less optimized for speed. The language-agnostic design enables testing diverse implementations without the need to modify the fuzzer. Unlike white-box approaches that require per-language integration (e.g., ASAN for C/C++, Miri for Rust), our approach works with any server implementation that follows the WebTransport specification and outputs structured logs.

3.3.3 Suggestions for Improvements

We propose the following improvements for future work:

- Extension to additional WebTransport implementations
- Client-side fuzzing in browsers
- Integration of coverage-guided feedback when source access is available

- Longer fuzzing campaigns with larger seed corpora

Conclusion

This thesis addressed fuzzing of the WebTransport protocol with the goal of identifying bugs in implementations and vulnerabilities. We presented a developed fuzzer based on BooFuzz that includes message structure mutations and sequence duplication.

The key achievements of this thesis include:

- Development of a universal fuzzer for the WebTransport protocol with aioquic integration for QUIC support
- Application of mutation and sequence duplication techniques tailored for WebTransport
- Analysis of results revealing bugs in implementations, including a reachable assertion bug in the Rust wtransport library
- Demonstration of the effectiveness of the black/gray-box approach in detecting worker thread panics

Our contribution strengthens protocol testing and contributes to the reliability of WebTransport at an early stage of its lifecycle. The thesis reflects important lessons about the significance of a language-agnostic approach when testing evolving standards.

The broader impacts on network security underscore the need for early fuzzing of new web technologies to prevent vulnerabilities in widely adopted systems. Since WebTransport is already implemented in major browsers but is still being actively developed, timely identification and fixing of bugs is critical for its safe deployment.

Future work could include extending the fuzzer with smarter mutation strategies leveraging machine learning, client-side fuzzing in browsers, and application to additional implementations of the protocol. The source code of the fuzzer is available at `https://github.com/qbitroot/webtransport-fuzzer`.

Bibliography

[1] BooFuzz Contributors. BooFuzz documentation, 2024. Available at https://boofuzz.readthedocs.io/en/stable/index.html.

[2] J. Chen et al. A survey of protocol fuzzing. *arXiv preprint*, 2024. arXiv:2401.01568v2. Available at https://arxiv.org/pdf/2401.01568v2.

[3] L. Daniele et al. LibAFL*: Advanced fuzzing for embedded systems. In *Proceedings of the Conference on Security*, 2024. Available at https://conand.me/publications/daniele-libaflstar-2025.pdf.

[4] Biagio Festa. wtransport GitHub repository, 2024. Available at https://github.com/BiagioFesta/wtransport.

[5] IETF. WebTransport over HTTP/3, 2024. IETF Draft. Available at https://datatracker.ietf.org/doc/draft-ietf-webtrans-http3/.

[6] X. Li et al. Fuzzing techniques for IoT and network protocols: A survey. *arXiv preprint*, 2024. arXiv:2503.19402. Available at https://arxiv.org/pdf/2503.19402.

[7] Joshua Pereyda. BooFuzz GitHub repository, 2024. Available at https://github.com/jtpereyda/boofuzz/.

[8] W3C. W3C WebTransport echo server example, 2024. Available at https://github.com/w3c/webtransport/blob/main/samples/echo/py-server/ipv4_echo_server.py.

[9] W3C. WebTransport API, 2024. W3C Recommendation. Available at https://www.w3.org/TR/webtransport/.

[10] Y. Wang et al. Enhancing network protocol fuzzing with AI-driven mutations. *Computers & Security*, 2024. DOI: 10.1016/j.cose.2025.104683. Available at https://doi.org/10.1016/j.cose.2025.104683.

Appendix A: Contents of the Online Appendix

The online appendix attached to this work contains the source code and data files used in the analysis. The source code is available at `https://github.com/qbitroot/webtransport-fuzzer`.

The repository contains:

- `main.py` — CLI entry point and BooFuzz session setup
- `src/` — fuzzer core: capsule definitions, sequence mutator, connection handler, WebTransport client, health check monitor, request logger, and server manager
- `analyze_logs.py` — offline log correlation tool
- `get-server-version.py` — server draft-version fingerprinting tool
- `server/aioquic/` — Python echo server (draft-03)
- `server/wtransport/` — Rust echo server (draft-09)

To run the fuzzer, use `uv run python main.py` with the appropriate flags as described in the project README.

Appendix B: User Manual

Prerequisites

The fuzzer requires Python 3.10+ and the `uv` package manager. For the Rust echo server, a Rust toolchain is needed. TLS certificates (self-signed) are used for local testing.

Basic Usage

```
# Validate connection
uv run python main.py --no-fuzz \
    --url https://127.0.0.1:6161/echo

# One-shot capsule fuzzer
uv run python main.py --mode oneshot \
    --url https://127.0.0.1:6161/echo

# Multistep scenario fuzzer
uv run python main.py --mode multistep \
    --url https://127.0.0.1:6161/echo

# With server subprocess management
uv run python main.py --mode multistep \
    --url https://127.0.0.1:6161/echo \
    --server-cmd "uv run server/aioquic/server.py \
        cert.pem key.pem 2>&1 | tee server.log"

# Resume from a specific test case index
uv run python main.py --mode multistep \
    --start-index 42 --end-index 100
```

Server Fingerprinting

Before fuzzing, identify which draft generation the target implements:

```
uv run get-server-version.py --no-verify \
    127.0.0.1 6161
```

Analyzing Results

After a fuzzing run, correlate test cases with server output:

```
uv run analyze_logs.py --log server.log \
    --db boofuzz-results/run_<timestamp>.db
```

The BooFuzz web UI is available at `http://localhost:26000` during a run.